

PlatformEQ EML Viewer

Phase A Telemetry — Implementation Report

Generated June 23, 2026 · Extension v0.1.0

Executive summary

Phase A adds a privacy-safe usage diary to the Chrome extension. When team members open the tool, import an email, or view an attachment, the extension records a short note about *that action* — not about the email content. Notes are saved locally in the browser until a future phase sends them to GCP.

Important: No network calls are made in Phase A. The extension still has zero host_permissions. Events accumulate in local storage only.

What changed at a glance

- Core module: `src/lib/telemetry.js` (queue, bucketing, SHA-256 fingerprint)
- Manifest: `identity` and `identity.email` permissions
- Nine event types across service worker, import, viewer, attachment pages
- Dedicated QA page: `src/pages/debug/telemetry.html`
- Documentation: `HANDOFF.md`, `PLAN.md §8`, `docs/telemetry.md`
- 13 unit tests in `test/telemetry.test.js` (39 total in suite)

1. Goals and scope

Phase A implements the local half of the telemetry plan: capture identified usage events, buffer them on disk, and enforce strict privacy guardrails. Sending data to GCP (Phase B), periodic flush alarms, and HANDOFF governance updates are explicitly out of scope for this phase.

In scope (Phase A)

- Event tracking module with local queue
- Chrome profile identity (work email via Okta→Google)
- Instrumentation of all planned user flows
- Coarse bucketing helpers (file size, attachment count, MIME category)
- SHA-256 content_fingerprint on import events (repeat-file analytics)
- Forbidden-key validation to block PHI-risk fields
- Unit tests for pure helper functions

Deferred to later phases

- **Phase B:** GCP Cloud Run ingest, host_permissions, build-time URL/API key, periodic flush
- **Phase C:** Opt-out policy, team announcement before production flush (HANDOFF updated)
- **Phase D:** In-extension Okta login (only if too many unknown users)

2. Privacy model

The extension handles Epic/healthcare email locally. Telemetry must never capture message content or identifiers that could contain patient information.

Never collected

- Email subject, from/to/cc, body, or dates
- Filenames or attachment names
- mailId, subject text, or parsed email body
- Raw error messages (only stable error_code enums)

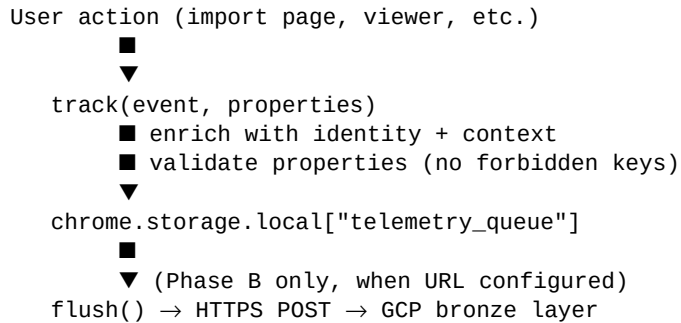
Safe to collect

- Corporate email from Chrome profile (chrome.identity.getProfileUserInfo)
- Extension version, Chrome version, OS platform
- Coarse buckets: file size band, attachment count band, MIME category
- Flow metadata: import method (drop vs picker), preview type, success/fail codes
- Parse duration in milliseconds
- content_fingerprint: SHA-256 hex of raw file bytes (same file → same hash)

The fingerprint is an opaque ID — not filename or subject — but linkable if someone holds the original file. A runtime guard (assertSafeProperties) rejects forbidden keys such as subject, filename, or body.

3. Architecture

Usage notes are written by extension pages and the background service worker. All notes land in a single array stored under the key `telemetry_queue` in `chrome.storage.local`, which persists across browser restarts.



Storage layout

Key	Storage	Purpose
telemetry_queue	chrome.storage.local	Array of event envelopes awaiting flush
telemetry_install_id	chrome.storage.local	Stable UUID per extension install
telemetry_session_id	chrome.storage.session	UUID per browser session (clears on quit)

Queue limits

- Maximum 500 events — oldest dropped on overflow
- Maximum age 7 days — stale events pruned on each write
- `track()` never throws — failures log to `console.debug` only

4. Event envelope

Every tracked event is wrapped in a consistent JSON envelope before being appended to the queue:

```
{
  "event": "import_succeeded",
  "ts": "2026-06-23T12:00:00.000Z",
  "user_email": "jane@company.com",
  "identity_source": "chrome",
  "install_id": "550e8400-e29b-41d4-a716-446655440000",
  "session_id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "extension_version": "0.1.0",
  "chrome_version": "131.0.6778.86",
  "platform": "MacIntel",
  "properties": {
    "content_fingerprint": "8f3a2b1c...",
    "method": "drop",
    "parse_ms": 142,
    "body_type": "html",
    "attachment_count_bucket": "1-3",
    "inline_image_count": 2,
    "has_pdf": false
  }
}
```

Identity (Scenario A — confirmed)

Users sign into Chrome with their work Google account (federated through Okta). The extension reads `user_email` from the Chrome profile — no Okta API calls. If Chrome is not signed in, events use `user_email: unknown` and `identity_source: unknown`, with a stable `install_id` as fallback.

5. Events instrumented

Event	Trigger	Key properties
<code>extension_opened</code>	Toolbar icon clicked	—
<code>session_reconciled</code>	Background prune on startup/install	<code>pruned_count</code>
<code>import_rejected</code>	Wrong extension or file too large	<code>reason: bad_extension too_large</code>
<code>import_started</code>	File accepted, parse begins	<code>method</code> , <code>file_ext</code> , <code>file_size_bucket</code> , <code>content_fingerprint</code>
<code>import_succeeded</code>	Mail saved and viewer opened	<code>parse_ms</code> , <code>body_type</code> , <code>attachment_count_bucket</code> , <code>inline_image_count</code> , <code>has_pdf</code>
<code>import_failed</code>	Parse or storage error	<code>error_code</code> , <code>content_fingerprint</code> (if bytes read)
<code>viewer_opened</code>	Viewer page loads	<code>body_type</code> , <code>attachment_count_bucket</code>
<code>attachment_opened</code>	Attachment tab loads	<code>preview_type</code> , <code>mime_category</code>
<code>attachment_downloaded</code>	Download link clicked	<code>mime_category</code>

6. Files changed

New files

File	Description
src/lib/telemetry.js	track(), flush(), bucketing, computeContentFingerprint(), queue
src/pages/debug/telemetry.html	Dedicated local queue inspector (QA)
test/telemetry.test.js	13 unit tests for helpers, fingerprint, forbidden-key guard
docs/telemetry.md	Full telemetry reference
HANDOFF.md	Install guide + telemetry summary for IT/compliance

Modified files

File	Changes
manifest.json	Added permissions: identity, identity.email (no host_permissions)
src/background/service-worker.js	track extension_opened on toolbar click; track session_reconciled after prune
src/pages/import/import.js	Import events + content_fingerprint; link to debug page
src/pages/viewer/viewer.js	track viewer_opened with body_type and attachment_count_bucket
src/pages/attachment/attachment.js	track attachment_opened and attachment_downloaded

7. Bucketing helpers

Helper	Input	Output bands
fileSizeBucket(bytes)	File size in bytes	0-100kb, 100kb-1mb, 1-10mb, 10mb+
attachmentCountBucket(count)	Non-inline attachment count	0, 1-3, 4-10, 10+
mimeCategory(mimeType)	MIME type string	image/*, application/pdf, other
deriveBodyType({hasHtml, text})	Mail body flags	html, text, empty, both
safeFileExtension(name)	Filename	Extension only (e.g. .eml), never full name
computeContentFingerprint(bytes)	Raw file ArrayBuffer	64-char SHA-256 hex

8. Testing and verification

Automated tests

Run `npm test`. The telemetry test file covers bucketing, body-type derivation, content fingerprint stability, safe file extension extraction, forbidden-key rejection, and queue pruning. Total: 39 tests, all passing.

Manual verification in Chrome

- 1. Reload the extension at `chrome://extensions`
- 2. Open `chrome-extension://<id>/src/pages/debug/telemetry.html` (or footer link on import page)
- 3. Use the tool: click icon, import a .eml twice, open an attachment
- 4. Refresh the debug page — confirm `content_fingerprint` matches on repeat imports
- 5. Confirm no network requests in DevTools Network tab

Build output

`npm run build` produces a bundled `telemetry-*.js` chunk (~4.4 KB gzip ~1.9 KB) shared across import, viewer, attachment, and service worker pages.

9. Next steps (Phase B)

- Deploy Cloud Run ingest endpoint in the GCP/medallion infra repo
- Land raw events in bronze (GCS or BigQuery — match existing medallion pattern)
- Add `VITE_TELEMETRY_URL` and `VITE_TELEMETRY_API_KEY` to extension build/CI
- Add scoped `host_permissions` for the Cloud Run URL only
- Wire `chrome.alarms` in the service worker for periodic flush (~15 min) and flush on startup
- Add silver/gold transforms for dashboards (DAU, import success rate, drop vs picker ratio)